

Лабораторная работа №4

“Бэггинг и бустинг. Метод случайного леса, AdaBoost и градиентный бустинг”

Ансамблевые методы машинного обучения

Ансамблевые методы — это подход в статистике и машинном обучении, в котором несколько **базовых моделей** (также называемых *слабыми моделями*, *слабыми учениками*) объединяются вместе для получения более точных и стабильных предсказаний, чем каждая из них по отдельности.

Существует три основных типа ансамблевых методов в машинном обучении:

- *Бэггинг (Bagging, Bootstrap Aggregating)*
- *Бустинг (Boosting)*
- *Стекинг (Stacking)*

Бэггинг и бустинг будут подробнее рассмотрены ниже.

Стекинг — ансамблевый метод, который комбинирует предсказания нескольких разных моделей (называемых базовыми моделями) с помощью модели мета-уровня (или мета-модели), которая обучается на их выходах, чтобы сделать финальное предсказание.

Разные модели по-разному обрабатывают данные и совершают разные ошибки, стекинг же позволяет использовать это разнообразие моделей и извлечь лучшее из каждой. Стекинг особенно эффективен, когда базовые модели независимы и разнородны.

В python ансамблевые методы можно найти в пакете [*sklearn.ensemble*](https://scikit-learn.org/) библиотеки *skikit-learn*.

Бэггинг

Бэггинг (*bootstrap aggregating, бутстрэп-агрегирование*) — ансамблевый алгоритм, предназначенный для улучшения стабильности и точности алгоритмов машинного обучения, используемых в задачах классификации и регрессии. Алгоритм *уменьшает дисперсию и помогает избежать переобучения*. Хотя он обычно применяется к методам машинного обучения на основе деревьев решений, его можно использовать с любым

видом моделей машинного обучения. Бэггинг является частным видом усреднения модели.

Основная идея бэггинга — обучить некоторое количество моделей одного и того же типа на разных случайных подвыборках исходных данных, а затем агрегировать их предсказания (усреднить для регрессии или выбрать ответ по принципу голоса большинства для классификации).

Подвыборки образуются из исходных данных (то есть из всех данных, которые имеются для решения задачи) путем применения *бутстрепа*. **Бутстреп** — это метод формирования подвыборок изначального датасета, где каждая подвыборка получается путем повторной выборки с возвращением элементов общего датасета.

Как пример можно привести задачу с корзиной шариков, на каждом из которых написано уникальное число. Пусть n — размер изначальной выборки (количество шариков в корзине), m — размер подвыборок, а T — число базовых моделей, используемых при решении методом бэггинга. Тогда алгоритм формирования одной подвыборки следующий: на каждой итерации бутстрепа из корзины случайным образом достаётся шарик, число на нем выписывается на листок и затем этот шарик возвращается в корзину; так повторяется m раз, пока не сформируется обучающая подвыборка. А уже этот алгоритм формирования подвыборки производится столько раз, сколько базовых моделей нам требуется обучить, то есть T раз.

Таким образом можно заключить, что при формировании подвыборок методом бутстрепа элементы выбираются случайным образом и равновероятно. Это также значит, что какой-то элемент из исходного набора данных может попасть в подвыборку несколько раз, а какой-то может ни разу.

Значение m размера подвыборки обычно **либо равно размеру изначальной выборки n , либо меньше его**. Если размер подвыборки равен размеру общего датасета $m = n$, то в каждую из подвыборок попадет примерно 63,2% уникальных элементов исходной выборки. Этот факт понадобится при оценке модели бэггинга.

Так почему же бэггинг эффективен при решении задач? Это происходит потому, что усреднение предсказаний от разных слабых моделей, обученных на случайных бутстреп-выборках, снижает дисперсию общей модели бэггинга.

Не приводя сложных формул, можно сказать, что дисперсия итоговой модели бэггинга будет следующей:

$$\sigma_{bag}^2 = \frac{\sigma^2}{T}$$

где T — количество базовых моделей в бэггинге, а σ^2 — их дисперсия. Предполагается, что модели независимы друг от друга и их дисперсии равны.

В конечном счете, такой подход делает итоговую модель менее чувствительной к шуму и переобучению.

Еще один важный нюанс бэггинга это то, что *базовые модели должны охватывать как можно больше возможных исходов для входного вектора x и формировать как можно более независимые ответы для него*. В этом смысле, обычные линейные алгоритмы (лабораторная №2) не очень подходят для этой задачи. Однако с этой задачей хорошо справляются решающие деревья.

Случайный лес

Случайный лес — это частный случай бэггинга, который использует деревья решений в качестве базовых моделей.

За счет своей приверженности к переобучению ответы деревьев получаются разнообразными и описывают самые разные исходы для входного x . Усреднение же ответа нивелирует переобучение каждой из моделей и конечное значение случайного леса получается достаточно точным и устойчивым к выбросам.

Для внесения еще большего разнообразия в ответы базовых деревьев, при их построении, предикаты в узлах формируются на основе m случайно отобранных признаков вектора x , где $m < n$, а n — общее число признаков вектора x . Отбор m случайных признаков происходит *для каждой вершины* строящегося дерева.

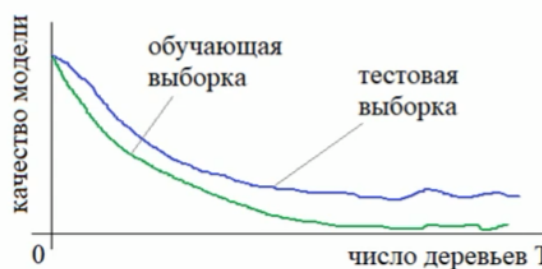
Таким образом деревья получаются очень случайными, как раз из этого такой метод и получил название *случайного леса*. Причем было показано, что наилучшие значения m следующие:

$$m = \lfloor \sqrt{n} \rfloor \text{ — для классификации}$$

$$m = \lfloor n/3 \rfloor \text{ — для регрессии}$$

Можно подвести итог, что при использовании модели случайного леса мы стремимся построить *глубокие и случайные деревья*, которые как можно более точно описывают свои обучающие подвыборки. Переобучение базовых моделей в таком случае является только плюсом, так как таким образом все вместе они наиболее разнообразно описывают входной вектор x .

Переобучение же случайного леса нивелируется усреднением при подсчете конечного ответа. Это также уменьшает количество гиперпараметров, которые надо подбирать при обучении. Причем рост такого гиперпараметра, как число базовых моделей, т.е. деревьев, не переобучает случайный лес:



Модель случайного леса [для классификации](#) и [для регрессии](#) можно найти в библиотеке *skikit-learn*.

Преимущества случайного леса:

- имеет высокую точность прогнозов (на большинстве задач работает лучше линейных алгоритмов); точность сравнима с точностью бустинга;
- практически не чувствителен к выбросам в данных из-за случайного сэмплирования выборок методом бутстрепа;
- не чувствителен к масштабированию (и вообще к любым монотонным преобразованиям) значений признаков, связано с выбором случайных подпространств;
- не требует тщательной настройки гиперпараметров, хорошо работает «из коробки»;
- способен эффективно обрабатывать данные с большим числом признаков и классов;
- редко переобучается, на практике добавление деревьев почти всегда только улучшает композицию (до определенного, предельного уровня);
- хорошо работает с пропущенными данными; сохраняет хорошую точность, даже если большая часть данных пропущена;

- могут быть расширены до неразмеченных данных, что приводит к возможности сделать кластеризацию и визуализацию данных, обнаруживать выбросы;
- легко распараллеливать и масштабировать (увеличивать число деревьев и их глубину).

Недостатки случайного леса:

- в отличие от одного дерева, результаты случайного леса сложнее интерпретировать;
- алгоритм работает хуже многих линейных методов, когда в выборке очень много разреженных признаков (тексты, Bag of words);
- случайный лес не умеет экстраполировать, в отличие от той же линейной регрессии;
- алгоритм склонен к переобучению некоторых задач, особенно на сильно зашумленных данных;
- для данных, включающих категориальные переменные с различным количеством уровней, случайные леса предвзяты в пользу признаков с большим количеством уровней: когда у признака много уровней, дерево будет сильнее подстраиваться именно под эти признаки, так как на них можно получить более высокое значение оптимизируемого функционала (информационный выигрыш);
- большой размер получающихся моделей. Требуется $O(N \cdot K)$ памяти для хранения модели, где K число деревьев.

Out-Of-Bag Error

При использовании бэггинга нам не требуется разделять изначальный датасет на обучающую и тестовую подвыборку. Однако нам каким-то образом нужно оценить работу обученной модели и для этого требуются данные не задействованные при обучении, для устранения предвзятости при оценивании.

Как было сказано ранее, подвыборки для обучения каждой конкретной базовой модели в беггинге формируются на основе метода бутстрепа. Если в изначальной выборке имеется n наблюдений, то вероятность выбора конкретного наблюдения на итерации бутстрепа равна $1/n$. А вероятность того, что наблюдение не выберут на итерации

$$P = 1 - \frac{1}{n}$$

Если же размер обучающей подвыборки также равен n , то вероятность не попадания наблюдения в обучающую подвыборку равна

$$P_{\text{не попад}} = (1 - \frac{1}{n})^n$$

при устремлении n к бесконечности, получим замечательный предел

$$\lim_{n \rightarrow \infty} (1 - \frac{1}{n})^n = \frac{1}{e} \approx 0.368$$

Из этого выражения следует, что вероятность не попадания наблюдения в обучающую подвыборку 36,8 % при стремлении размера выборки к бесконечности. Такие данные называют ***Out-Of-Bag*** для конкретной базовой модели. Из этого как раз и получено число 63,2% уникальных элементов исходной выборки, попадающее в обучающую подвыборку базовой модели.

Таким образом формирование оценки ***Out-Of-Bag Error*** происходит по следующим шагам:

1. Получение предсказаний на каждой базовой модели для ее *Out-Of-Bag* набора данных;
2. Агрегация этих ответов для каждого объекта из общей обучающей выборки: мы собираем предсказания всех деревьев, для которых объект был ООВ, и агрегируем предсказания для него (голосование / усреднение).
3. Сравниваем с реальной меткой.

Out-Of-Bag Error — это оценка усредненных выходов базовых моделей бэггинга для наблюдений, на которых они не обучались.

Если задать параметр ***oob_score=True*** при создании модели бэггинга

```
model = RandomForestClassifier(oob_score=True)
```

то после обучения можно получить предсказанные для ООВ значения и Ассигуру для них из соответствующих полей:

```
out_of_bag_predictions = model.oob_prediction_  
print("OOB Accuracy:", model.oob_score_  
print("OOBE: ", 1 - model.oob_score_)
```

Бустинг

Бустинг (boosting) — это ансамблевый метод, при котором слабые модели обучаются последовательно, и каждая новая модель пытается исправить ошибки предыдущих. В отличие от бэггинга, где модели обучаются независимо и их предсказания усредняются, в бустинге модели «учатся на ошибках» друг друга.

Однако вернемся к бэггингу. Гипотезу конечной модели бэггинга, которая усредняет ответы T моделей, можно представить в следующем виде:

$$a_{Beg}(x) = \frac{1}{T} \sum_{i=1}^T a_i(x)$$

Эффективность бэггинга основана на независимости T алгоритмов и статистической независимости их ответов. Однако на практике алгоритмы $a_1(x)$, $a_2(x)$, ..., $a_T(x)$ все же выдают несколько зависимые ответы (в общем случае, полной независимости достичь невозможно). Также можно заметить, что в бэггинге ответы моделей взвешиваются с одинаковым коэффициентом $1/T$, то есть значимость каждой модели полагается равной. Это означает, что мы не достигаем максимального качества при композиции алгоритмов.

Поэтому в 1995 году Йоав Фройнд и Роберт Шапир предложили более общий вариант для композиции алгоритмов:

$$a(x) = \sum_{t=1}^T \alpha_t b_t(x)$$

где α_t — это вес для ответа слабой модели $b_t(x)$.

Причем алгоритм $b_t(x)$ возвращает **вещественное значение** — $b_t(x) \in \mathbb{R}$. В задачах же классификации каждая из слабых моделей будет искать значение, от которого зависит выдаваемая метка, как например отступ от разделяющей плоскостности или вероятность отнесения к классу.

Таким образом алгоритмы $b_t(x)$ выдают не жесткие конечные решения, а более точные промежуточные — вещественные. В этом случае, композиция таких алгоритмов приводит к лучшим результатам.

Задача же сводится к нахождению оптимальных весовых коэффициентов α_t и моделей $b_t(x)$, где $t \in [1, T]$. Под нахождением оптимальной модели $b_t(x)$ подразумевается обучение этой модели таким

образом, чтобы она, в композиции с другими моделями, наилучшим образом решала общую задачу. Для достижения этих целей вводят оптимизируемую **функцию стоимости Q , которая оценивает работу композиции слабых алгоритмов**. Например для задачи бинарной классификации можно записать следующую функцию стоимости, подсчитывающую количество неверных классификаций:

$$Q(\alpha, b) = \sum_{i=1}^l [M_i < 0] = \sum_{i=1}^l [y_i \cdot a(x_i) < 0] = \sum_{i=1}^l [y_i \cdot \sum_{t=1}^T \alpha_t b_t(x_i) < 0] \rightarrow \min$$

Однако оптимизация данной функции, включающая одновременный подбор T коэффициентов α_t и T моделей $b_t(x)$, является очень сложной задачей. Поэтому вышеупомянутыми исследователями были предложены две эвристики:

- «Жадное» построение композиции — поиск (выбор) текущего алгоритма $b_t(x)$ и весового множителя α_t при фиксации ранее найденных алгоритмов и весовых множителей: $b_1(x), \dots, b_{t-1}(x)$ и $\alpha_1, \dots, \alpha_{t-1}$. То есть, на каждом шаге t мы вычисляем только один весовой коэффициент α_t и обучаем только один алгоритм $b_t(x)$ преимущественно на тех образах обучающей выборки, на которых предыдущие алгоритмы показали слабые результаты.
- Аппроксимировать исходную функцию стоимости гладкой дифференцируемой функцией потерь, чтобы мы могли численно или аналитически решать оптимизационную задачу (было описано в лабораторной работе №2, в части про аппроксимацию $L(w) = [M < 0]$).

Эти два подхода положены в основу практически всех алгоритмов **бустинга**. То есть каждая следующая добавляемая в композицию модель пытается решить задачу для тех наблюдений, с которыми плохо справились предыдущие модели, как бы “бустит” весь ансамбль.

В качестве слабой модели в **бустинге** обычно выбирается алгоритм, который модно быстро обучить. Как например решающее дерево с одним узлом и двумя листьями (“пень”).

AdaBoost

Adaptive Boosting — алгоритм предложенный Фройндом и Шапиром изначально для решения задачи бинарной классификации с метками 1 и -1 .

Для аппроксимации *функции потерь* они предложили использовать *экспоненциальную функцию потерь*:

$$Q(\alpha_t, b_t(x)) = \sum_{i=1}^l [M_i < 0] \leq \sum_{i=1}^l \exp(-M_i) = \sum_{i=1}^l \exp(-y_i \cdot \sum_{t=1}^T \alpha_t b_t(x_i)) \rightarrow \min$$

Выделив последнюю итерацию алгоритма, Q можно расписать как:

$$Q(\alpha, b) = \sum_{i=1}^l \exp(-y_i \cdot \sum_{t=1}^{T-1} \alpha_t b_t(x_i)) \exp(-y_i \cdot \alpha_T b_T(x_i))$$

Причем, так как мы используем “жадный” алгоритм, то для последнего шага T значения коэффициентов $\alpha_1, \dots, \alpha_{T-1}$ и моделей $b_1(x_i), \dots, b_{T-1}(x_i)$ уже будут вычислены на предыдущих итерациях. Таким образом их можно выразить такое фиксированное значение как

$$\omega_{i, T-1} = \exp(-y_i \cdot \sum_{t=1}^{T-1} \alpha_t b_t(x_i)), i = 1, \dots, l$$

Оно вычисляется для каждого i -го наблюдения из обучающей выборки, поэтому его можно назвать **весом для i -го наблюдения из обучающей выборки**. Из формулы видно, что чем больше *отступ* i -го наблюдения уходит в отрицательное значение, то тем больше становится весовой коэффициент этого наблюдения.

Выделяя шаги $T-1$, $T-2$ и т.д. мы получим итеративный алгоритм, где в конце итерации $t-1$ пересчитываются веса для наблюдений, которые используются на следующем шаге t . Таким образом при обучении базовой модели $b_t(x)$ мы будем акцентировать внимание на объектах с большими ω_i , так как они плохо классифицируются моделями с предыдущих шагов.

Это соответствует основной идее бустинга — **последовательно обучать модели на объектах, на которых предыдущая композиция моделей показала наихудшие результаты**.

Также эти веса нормируют:

$$\omega_{sum} = \sum_{i=1}^l \omega_i; \omega_i = \omega_i / \omega_{sum}, i = 1, \dots, l$$

чтобы в итоге сумма коэффициентов была равна 1.

Для обучения же оптимальной модели $b_t(x)$ на шаге t , решается задача оптимизации следующей функции стоимости:

$$N(b_t(x)) = \sum_{i=1}^l \omega_{i, t-1} [b_t(x_i) \neq y_i] \rightarrow \min$$

то есть, минимизируя эту функцию, модель $b_t(x)$ стремится не ошибаться на наблюдениях с большими ω_i .

Исследователи также доказали, что при решении задачи с использованием экспоненциальной функцией потерь, лучшим коэффициентом α_t для модели t будет:

$$\alpha_t = \frac{1}{2} \ln \frac{1 - N(b_t(x))}{N(b_t(x))}$$

Систематизируя все вышеописанное, кратко опишем алгоритм **AdaBoost** для классификации:

1. Задаем начальные веса для объектов: $\omega_{i, 0} = 1/l$, $i = 1, \dots, l$, где l — количество объектов в обучающей выборке.
2. Для всех $t = 1, \dots, T$:
 - 2.1. Найти наилучшую модель $b_t(x)$, по условию

$$N_t(b_t(x)) = \sum_{i=1}^l \omega_{i, t-1} [b_t(x_i) \neq y_i] \rightarrow \min$$

- 2.2. Вычислить α_t

$$\alpha_t = \frac{1}{2} \ln \frac{1 - N(b_t(x))}{N(b_t(x))}$$

- 2.3. Обновить веса объектов

$$\omega_{i, t} = \omega_{i, t-1} \cdot \exp(-y_i \cdot \alpha_t b_t(x_i)), i = 1, \dots, l$$

- 2.4. Нормировать веса

$$\omega_{sum} = \sum_{i=1}^l \omega_{i, t}; \omega_{i, t} = \omega_{i, t} / \omega_{sum}, i = 1, \dots, l$$

AdaBoost в задаче регрессии

Для задачи регрессии мы должны выбрать другую функцию стоимости для оценки композиции моделей бустинга. В качестве таковой можно выбрать сумму квадратических ошибок по всем наблюдениям:

$$Q(\alpha, b) = \sum_{i=1}^l (y_i - a(x_i))^2 \rightarrow \min$$

Расписав ее по тому же принципу, что и в задаче классификации, получим:

$$Q(b) = \sum_{i=1}^l (y_i - \sum_{t=1}^T \alpha_t b_t(x_i))^2 = \sum_{i=1}^l (y_i - \sum_{t=1}^{T-1} b_t(x_i) - b_T(x_i))^2$$

причем **в задаче регрессии** коэффициент α_t можно сократить, так как это *масштабный множитель*, и модель $b_t(x)$ может рассчитывать свое выходное значение уже с его учетом.

Из-за жадности алгоритма, аналогично классификации, на шаге T обозначим фиксированную часть как:

$$S_{i, T-1} = y_i - \sum_{t=1}^{T-1} b_t(x_i), i = 1, \dots, l$$

Это значение можно назвать **остатком**. Так, i -ый *остаток* на шаге t соответствует тому, чего не хватило сумме выходов обученных до этого моделей, чтобы получить истинное значение для наблюдения x_i .

Тогда на шаге t можно обучить такую модель $b_t(x)$, которая бы своими выходами устраняла этот остаток. То есть для обучения базовой модели $b_t(x)$ минимизировать следующую функцию стоимости:

$$N_t(b_t(x)) = \sum_{i=1}^l (S_{i, t-1} - b_t(x_i))^2 \rightarrow \min$$

Резюмируя все вышеописанное, кратко опишем алгоритм бустинга для регрессии:

1. Начальная инициализация остатков $S_{i, 0} = y_i, i = 1, \dots, l$
2. Для всех $t = 1, \dots, T$:

- 2.1. Найти наилучшую модель $b_t(x)$, которая минимизирует остатки $S_{i, t-1}$
- 2.2. Обновить остатки

$$S_{i, t} = S_{i, t-1} - b_t(x_i)$$

Подытожив скажем, что для обучаемой на шаге t модели $b_t(x)$ целевыми значениями являются не истинные значения y_i , $i = 1, \dots, l$, а *остатки* $S_{i, t-1}$ с предыдущей итерации алгоритма, то есть:

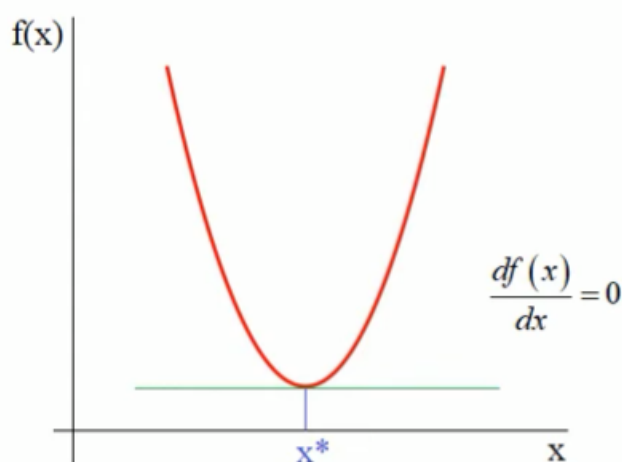
$$b_t(x_i) \rightarrow S_{i, t-1}$$

Такой подход, с заменой целевых значений на *остатки* для каждого последующего слабого алгоритма в композиции бустинга, отчасти соответствует алгоритму *градиентного бустинга*. Но чтобы его лучше понять, для начала обсудим *градиентный спуск*.

Градиентный спуск

Оптимизация (в математике, информатике и исследовании операций) — задача нахождения экстремума (минимума или максимума) целевой функции в некоторой области конечномерного векторного пространства, ограниченной набором линейных и/или нелинейных равенств или неравенств. Чаще всего эту задачу решают для нахождения минимума функции.

Известно, что в производная в точке экстремума некоторой функции равна нулю, то есть на примере одномерной функции:



Такую задачу можно решить аналитически сразу отыскав нужную точку минимума, однако с увеличением количества аргументов (длины

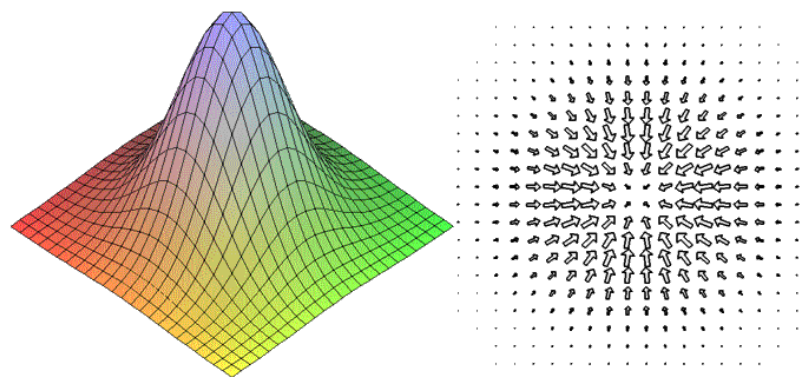
вектора x) сложность задачи резко возрастает. Поэтому на замену сложным аналитическим методам были придуманы численные методы приближения к минимуму, одним из которых является *метод градиентного спуска*.

Метод градиентного спуска — это численный метод итерационного приближения к минимуму функции с помощью ее *градиента*. В свою очередь, **градиент** — **вектор**, своим направлением указывающий направление *наискорейшего* роста функции в некоторой ее точке. Таким образом, чтобы найти минимум требуется двигаться в противоположную росту функции сторону, то есть в сторону **антиградиента**.

В общем случае задача решается для некоторой n -мерной функции $f(x)$, где $x = [x_1, x_2, \dots, x_n]$. Частная производная по некоторому аргументу x_i функции указывает **направление роста этого аргумента** x_i в некоторой точке. В точке **экстремума этого аргумента** значение производной будет равно 0. Из этого следует, что компонентами вектора градиента для некоторой точки x будут значения частных производных в этой точке по каждому из аргументов:

$$\text{grad}_f(x) = \left[\frac{\partial f(x)}{\partial x_1}, \frac{\partial f(x)}{\partial x_2}, \dots, \frac{\partial f(x)}{\partial x_n} \right]$$

Для двумерной функции (слева) векторы градиентов в различных ее точках (справа) будут выглядеть следующим образом:



Причем, так как градиент указывает направление **наискорейшего роста**, то на этом рисунке вектор градиента в точке тем больше, чем круче в ней подъем горки.

Как было указано ранее, градиентный спуск — это пошаговый алгоритм приближения к минимуму. Это значит, что алгоритм стартует с некой точки x и на каждом шагу изменяет ее для получения все лучшего результата. В градиентном спуске для этого используется антиградиент, который с каждым шагом приближает нашу точку к минимуму.

На шаге t алгоритма градиентного спуска следующее приближение к минимуму $x^{(t)}$ находится следующим образом:

$$x^{(t)} = x^{(t-1)} - \lambda_{t-1} \cdot \text{grad}(x^{(t-1)}) =$$

Или расписав для каждого компонента точки x^t :

$$x_1^{(t)} = x_1^{(t-1)} - \lambda_{t-1} \cdot \frac{\partial f(x^{(t-1)})}{\partial x_1}$$

$$x_2^{(t)} = x_2^{(t-1)} - \lambda_{t-1} \cdot \frac{\partial f(x^{(t-1)})}{\partial x_2}$$

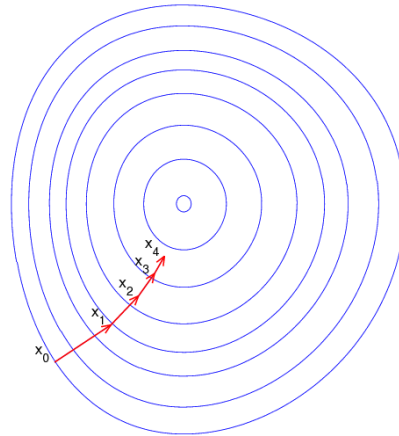
...

$$x_n^{(t)} = x_n^{(t-1)} - \lambda_{t-1} \cdot \frac{\partial f(x^{(t-1)})}{\partial x_n}$$

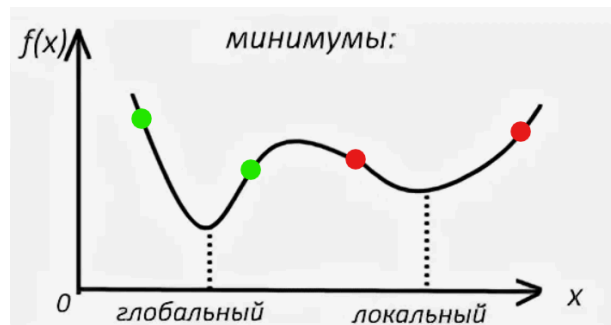
Коэффициент λ называется **шагом сходимости** и применяется для того, чтобы не “перепрыгнуть” минимум, так как значение градиента может быть очень большим. Шаг сходимости задается перед началом градиентного спуска и может изменяться по ходу выполнения алгоритма для контроля скорости обучения.

Сам же алгоритм продолжается, пока не достигнет точки останова. Часто как точку останова используют достижение заранее заданного максимального числа шагов или если изменение точки между итерациями стало меньше некоторого значения. Минимумом функции в таком случае считают последнюю полученную точку.

Графически решение для двумерного пространства выглядит следующим образом (нижний индекс обозначает номер шага):



Однако у градиентного спуска есть недостаток — застревание в локальных минимумах, которые не в полной мере являются ответом на задачу. Например, если в качестве начальной точки мы выберем одну из зеленых, то попадем в глобальный минимум, красную — в локальный:



Поэтому этот алгоритм имеет множество усовершенствований, которые позволяют быстрее находить минимум, делать это точнее, не застревать в локальных минимумах и т.д..

Градиентный бустинг

Вернемся немного назад, *гипотеза* для модели бустинга следующая:

$$a(x) = \sum_{t=1}^T \alpha_t b_t(x)$$

Теперь запишем *функцию стоимости* для бустинга в общем виде:

$$Q(\alpha, b) = \sum_{i=0}^l L(a(x_i), y_i) \rightarrow \min$$

Как было описано ранее, в алгоритме AdaBoost применяются следующие **функции потерь**:

$$L(a(x), y) = \exp(-y a(x)) \text{ — для классификации}$$

$$L(a(x), y) = (y - a(x))^2 \text{ — для регрессии}$$

Однако это не единственные функции потерь, которые можно использовать. Тогда возникает вопрос: существует ли общее решение бустинга для произвольной функции потерь?

Ответ на этот вопрос в 1999 году дал Джером Фридман. Он разработал следующий общий алгоритм бустинга для **дифференцируемых функций потерь**:

Распишем функцию стоимости, подставив туда гипотезу:

$$Q(\alpha, b) = \sum_{i=1}^l L(a(x_i), y_i) = \sum_{i=1}^l L\left(\sum_{t=1}^T \alpha_t b_t(x_i), y_i\right) \rightarrow \min$$

Выделив шаг T получим:

$$Q(\alpha, b) = \sum_{i=1}^l L\left(\sum_{t=1}^{T-1} \alpha_t b_t(x_i) + \alpha_T b_T(x_i), y_i\right) \rightarrow \min$$

Обозначим сумму выходов моделей до шага T как

$$f_{T-1, i} = \sum_{t=1}^{T-1} \alpha_t b_t(x_i), \quad i = 1, \dots, l$$

и назовем **приближением композиции** для входного вектора x_i на шаге $T-1$. Так как алгоритм жадный, то на шаге T это будет уже **рассчитанное ранее фиксированное значение**. Совокупность же всех **приближений** на шаге $T-1$ для каждого вектора x_i из обучающей выборки размером l обозначим через вектор

$$f_{T-1} = [f_{T-1, 1}, f_{T-1, 2}, \dots, f_{T-1, l}]^T$$

Приближения после шага T :

$$f_{T, i} = \sum_{t=1}^T \alpha_t b_t(x_i), \quad i = 1, \dots, l$$

$$f_T = [f_{T,1}, f_{T,2}, \dots, f_{T,l}]^T$$

Также перепишем функцию стоимости:

$$Q(f_T) = \sum_{i=1}^l L_i(f_{T,i}, y_i) \rightarrow \min$$

Теперь такое представление легче свести к **градиентному спуску**. Как видно, функция стоимости Q зависит от точки приближения f_T . После шага $T-1$ имеем точку f_{T-1} , значит следуя методу градиентного спуска на шаге T для приближения к минимуму Q мы должны найти следующее значение f_T используя антиградиент:

$$f_T = f_{T-1} - \alpha g,$$

$$f_{T,i} = f_{T-1,i} - \alpha g_i, \quad i = 1, \dots, l$$

где компоненты градиента считаются по следующей формуле:

$$g_i = \frac{\partial Q(f_{T-1})}{\partial f_{T-1,i}} = \frac{\sum_{k=1}^l \frac{\partial L(f_{T-1,k}, y_k)}{\partial f_{T-1,i}}}{\partial f_{T-1,i}} = \left[\frac{\partial L(f_{T-1,k}, y_k)}{\partial f_{T-1,i}} = 0, \text{ если } k \neq i \right] = \frac{\partial L(f_{T-1,i}, y_i)}{\partial f_{T-1,i}}, \quad i = 1, \dots, l,$$

Ещё раз взглянув и расписав формулу для $f_{T,i}$

$$f_{T,i} = \sum_{t=1}^T \alpha_t b_t(x_i) = \sum_{t=1}^{T-1} \alpha_t b_t(x_i) + \alpha_T b_T(x_i) = f_{T-1,i} + \alpha_T b_T(x_i)$$

увидим, что

$$f_{T-1,i} - \alpha g_i = f_{T-1,i} + \alpha_T b_T(x_i)$$

Из этого следует, что в общем случае на шаге обучения t модель $b_t(x)$ должна выдавать выходные значения как можно ближе к антиградиенту, чтобы приблизить функцию потерь Q к минимуму.

Таким образом, модель $b_t(x)$ обучается с учетом функции стоимости

$$N_t(b_t(x)) = \sum_{i=1}^l (-g_i - b_t(x_i))^2 = \sum_{i=1}^l (g_i + b_t(x_i))^2 \rightarrow \min$$

Также иногда **антиградиент** на шаге t называют **псевдо-остатком**, так как он похож на остаток в бустинге для регрессии.

После нахождения $b_t(x)$ остается только вычислить весовой коэффициент α_t для этой модели. Для этого решается задача одномерной оптимизации

$$A_t(\alpha_t) = \sum_{i=0}^l L(f_{t-1,i} + \alpha_t b_t(x_i), y_i) \rightarrow \min$$

При этом существуют достаточно простые методы для нахождения подходящего коэффициента и эта подзадача не является вычислительно трудоемкой.

Суммируя вышеописанный алгоритм, его можно кратко описать так:

1. Инициализация вектора f_0 . Начальные же значения *приближений* зависят от решаемой задачи, но зачастую это:
 - для регрессии $f_{0,i} = 0, i = 1, \dots, l$;
 - для бинарной классификации $f_{0,i} = p, i = 1, \dots, l$, где p — доля объектов целевого класса в обучающей выборке.
2. Для всех $t = 1, \dots, T$:
 - 2.1. Найти градиент в точке приближения f_{t-1}

$$g_i = \frac{\partial L(f_{t-1,i}, y_i)}{\partial f_{t-1,i}}, i = 1, \dots, l$$

- 2.2. Найти наилучший алгоритм $b_t(x)$ с выходными значениями близкими к антиградиенту

$$N_t(b_t(x)) = \sum_{i=1}^l (g_i + b_t(x_i))^2 \rightarrow \min$$

- 2.3. Найти весовой коэффициент, найдя минимум

$$A_t(\alpha_t) = \sum_{i=0}^l L(f_{t-1,i} + \alpha_t b_t(x_i), y_i) \rightarrow \min$$

- 2.4. Обновить вектор приближений

$$f_{t,i} = f_{t-1,i} + \alpha_t b_t(x_i), i = 1, \dots, l$$

Использование градиентного спуска при построении композиции моделей и дало название этому алгоритму — **градиентный бустинг (gradient boosting)**.

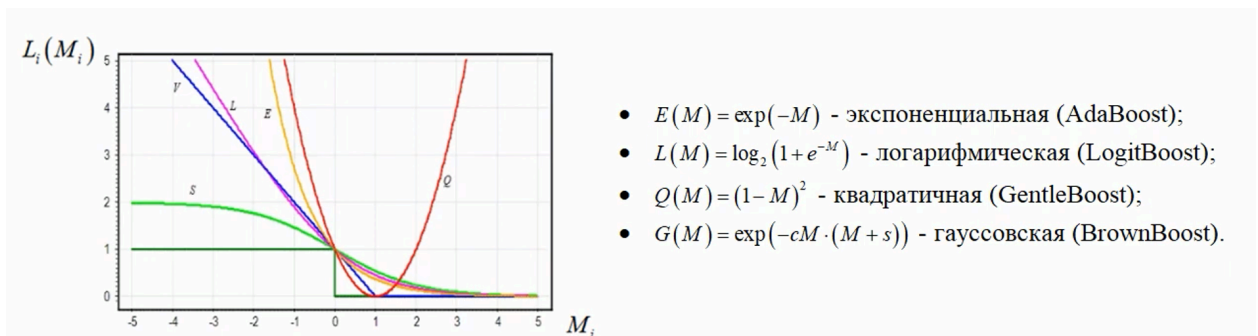
Также стоит упомянуть **стохастический градиентный бустинг (Stochastic Gradient Boosting)**. В отличие от стандартного градиентного бустинга, где антиградиент считает по всем наблюдениям из обучающей выборки, в этом алгоритме на каждом шаге t для этих целей используется случайная подвыборка из наблюдений. Это позволяет ускорить вычисления, а также уменьшить чувствительность к выбросам.

Функции потерь

При решении задачи регрессии задействуют следующие:

- $L(a(x_i), y_i) = (a(x_i) - y_i)^2$ — квадратичная ошибка
- $L(a(x_i), y_i) = |a(x_i) - y_i|$ — абсолютная ошибка
- $L(a(x_i), y_i) = \{\beta|a(x_i) - y_i|, \text{ для } a(x_i) - y_i < 0; (1 - \beta)|a(x_i) - y_i|, \text{ для } a(x_i) - y_i \geq 0\}$ — *Quantile loss*, позволяющая использовать квантиль β .

В задаче же классификации используются функции аппроксимирующие “ступеньку” $L(w) = [M < 0]$:



Можно заметить, что **AdaBoost для классификации и для регрессии являются частными случаями градиентного бустинга с экспоненциальной и квадратичной функциями потерь соответственно.**

Посмотрев еще раз на веса для наблюдений в AdaBoost, увидим:

$$\omega_{i, t-1} = \exp(-y_i \cdot \sum_{t=1}^{T-1} \alpha_t b_t(x_i)) = \exp(-y_i \cdot f_{t-1, i}), i = 1, \dots, l$$

Что значит: что чем хуже было приближение для i -го наблюдения на предыдущих шагах, тем больший вес оно получит на шаге t .

В AdaBoost для регрессии посмотрим на градиент

$$g_i = \frac{\partial L(f_{t-1,i}, y_i)}{\partial f_i} = \frac{\partial ((y_i - f_{t-1,i})^2)}{\partial f_i} = -(y_i - f_{t-1,i}) = -(y_i - \sum_{k=1}^{t-1} b_k(x_i)) = -S_{i,t-1}$$

Таким образом антиградиент соответствует остатку в регрессии с квадратичной функцией потерь.

В целом, неважно какую задачу мы решаем градиентным бустингом, классификацию или регрессию, каждая из слабых моделей $b_i(x)$ решает свою задачу регрессии, где в качестве целевого значения выступает *антиградиент* функции потерь на основе предыдущих шагов, который можно обозначить как *псевдо-остаток*:

$$N_i(b_i(x)) = \sum_{i=1}^l (g_i + b_i(x_i))^2 = \sum_{i=1}^l (S_{i,t-1} - b_i(x_i))^2 \rightarrow \min$$

Конечный результат определяется на основе взвешенной суммы выходов базовых моделей:

$$a(x) = \sum_{t=1}^T \alpha_t b_t(x)$$

Модели бустинга AdaBoost [для классификации](#) и [для регрессии](#) и градиентного бустинга [для классификации](#) и [для регрессии](#) можно найти в библиотеке **skikit-learn**. А также можно использовать мощные инструменты из библиотеки [XGBoost](#).

Преимущества и недостатки бустинга

Преимущества:

- Обеспечивает очень высокую точность и часто выигрывает соревнования по машинному обучению.
- Поддерживает разные типы задач и функций потерь, включая классификацию, регрессию и ранжирование.
- На каждом шаге сосредотачивается на исправлении ошибок предыдущих моделей, что улучшает итоговое качество.

- Обладает высокой гибкостью за счёт множества настраиваемых гиперпараметров.
- Не требует нормализации или ONE преобразования признаков.
- Позволяет оценить важность признаков и влияние каждого из них на результат.

Недостатки:

- Интерпретация предсказаний затруднена по сравнению с простыми моделями.
- Модели обучаются последовательно, что делает бустинг медленным и ресурсоемким.
- Имеет склонность к переобучению.
- Чувствителен к шуму и выбросам, так как старается обучиться даже на ошибочных данных.
- Необходима большая по сравнению с другими алгоритмами обучающая выборка.
- Требуется тщательной настройки гиперпараметров для достижения хороших результатов.
- Может уступать по эффективности на разреженных или текстовых данных.

Задание

1. Использовать датасет, подготовленный на первой лабораторной работе.
2. Решить задачу классификации методом случайного леса. Оценить работу модели через ООВ данные.
3. Решить задачу классификации методом AdaBoost и методом градиентного бустинга.
4. Оценить работу моделей и построить ROC-кривые.
5. Выгрузить результат работы на Github